PERINKPHERAL

Application platform
Datasheet and development guide

Display	4.2" e-paper, 400 x 300 pixels, 1 bit
Input	4 buttons
Application	WebAssembly module and manifest
Language	C, no standard library
Interface	one header, sdk/host_api.h, ABI 2
Memory	64 KB per application
Build and upload	about 10 s, no reflashing

I Datasheet: parameters and execution model

II Development guide: package, screen, input, storage, network

III Reference: host API and SDK library

revision 2, 2026-11-14 / host API of sdk/host_api.h, abi 2 / preliminary: the host API is not frozen before the first release
github.com/GrakovNe/perinkpheral / MIT

Contents

Part I. Datasheet 4

1. Overview 4
   - Features 4
   - How an application runs 4

2. Characteristics 5
   - Display 5
   - Input 5
   - The sandbox 5
   - Storage 6
   - Network 6
   - Time and background work 6
   - Text 7

3. Execution model 7
   - Application lifecycle 7
   - Handler guarantees 7
   - State 8
   - Navigation and refresh 8

Part II. Development guide 9

4. A first application 9
   - Requirements 9
   - Create, build, run 9
   - Sample application 10
   - Build report 10

5. The package 11
   - The manifest 11
   - Polls 12
   - Settings 12

6. The screen 13
   - Surface and colour 13
   - Fonts 13
   - Text beyond ASCII 14
   - Pictures and icons 14
   - The kit 15
   - Requesting a refresh 16

7. Buttons 17
   - BACK 17
   - Lists, pickers, dates 18

8. Storage 18
   - The store 18
   - Files 18

9. The network 19
   - Values extracted by the host 19
   - Streamed answers 19

Requests from code 20
Failed requests 20
10. Background work 21
Notifications 21
Alarms, timers 21
11. Time 21
12. Languages 22
13. Design rules 22
Screen content 22
Memory 23
State and time 23
Interaction and data 23
14. Tools and debugging 24
Common problems 24
15. Publishing 25
Part III. Reference 26
16. Return codes 26
17. Exports 26
18. Drawing 27
19. The kit 28
20. Navigation, storage and network 29
21. Time 29
22. The system API 30
23. The SDK library 30
app_text.h: short strings without printf 30
app_values.h: what the host picked 31
app_ui.h: what the buttons do on the kit's screens 31
app_alloc.h, json_pick.h, and what is generated 31
24. Limits 31

Part I. Datasheet

1 Overview

perINKpheral is a desk display: a 4.2" black-and-white e-paper panel with four buttons, built on an ESP32-C6. It shows information that is read at a glance: the time, the weather, the next meeting, the state of a home.

The firmware is called the *host*. The host provides the menu and the settings. Everything else on the screen is an *application*. Applications are installed from a catalog without reflashing the device.

An application consists of a WebAssembly module and a manifest. The module is written in C against a single header, `sdk/host_api.h`. The host runs it in an interpreter (wasm3).

The host's own pages, Settings and About, are applications built with the same header.

Features

- Display: 400 x 300 pixels, 1 bit, dithered greys, partial refresh in about 0.5 s
- Input: four buttons; short press, long press, key down and key up events
- Sandbox: 64 KB of private memory; network access through the host only
- Network: HTTP and HTTPS requests performed by the host, as scheduled polls or on request from the code
- Data: the host extracts values from JSON answers; any answer can be streamed in 1 KB pieces
- Background work: answers, alarms and device events are delivered while the application is off the screen
- Storage: a key-value store and files, kept across restarts and reinstalls
- Design system: lists, notices, pickers, cells and a button row, drawn by the host
- Notifications: shown by the host over any screen
- Settings: declared in the manifest, edited on the device's web page
- Localization: strings per language, loaded by the host; Latin with diacritics, Cyrillic and Greek
- Tools: build, check and measure an application; control a device over the network

How an application runs

An application is a set of exported handlers with fixed names. The host calls a handler when an event occurs: a button press, a timer, incoming data, a request to draw.

A handler updates the application's state and requests a redraw. The host calls `app_on_draw()` when it needs a frame. The host decides when and how the panel is refreshed.

One application is loaded at a time. The host unloads it when it leaves the screen. Persistent state is kept in the application's store; global variables live until the unload.

2 Characteristics

Display

Parameter	Value	Notes
Panel	4.2" e-paper, SSD1683	no backlight; the picture stays without power
Resolution	400 x 300 px	about 120 px per inch
Surface of an application	400 x 260 px	under the host's header; 400 x 300 with `"fullscreen": true`
Depth	1 bit	17 levels of grey by a fixed 4 x 4 dither, for areas only
Partial refresh	0.46 - 0.53 s	only the changed rectangle is sent
Full refresh	2.2 s (2.7 s)	fast waveform (default waveform); the panel blinks
Full refresh after	5 partial ones	a setting of the user's; clears the traces partial refreshes leave
Refresh after a press	at once	
Refresh nobody asked for	at most 1 per 60 s	a setting of the user's (the panel's maker advises 180 s)
A row of pictures	about 1.5 per second	`HOST_REDRAW_NO_BLINK`; traces pile up until the row ends

Input

Parameter	Value	Notes
Buttons	4	in this order on the device: UP, OK, DOWN, BACK
Events of a button	4	key down, key up, pressed, long pressed
Long press	600 ms	reported while the button is still held
Reserved	long BACK	always the host's: empties the stack and shows the menu

The sandbox

Parameter	Value	Notes
Linear memory	64 KB, fixed	one wasm page: 8 KB of stack, static data, the rest for `app_alloc()`; it cannot grow
Module (`app.wasm`)	48 KB at most	executed from flash, not copied into memory
Interpreter's memory	64 KB	about 11 KB for any application plus 4 bytes for every byte of code that has run
Code that can all run	about 13 KB	follows from the line above; `mkapp` warns before the device fails
Time of one handler	3 s	one that runs longer is stopped, and the application with it
Instances in memory	1	of all applications together
Floating point	none	the build refuses a module that has a float operation

Parameter	Value	Notes
C library	none	`sdk/lib` instead: strings, numbers, an arena, `memcpy` and its family
Installed applications	6	per device; a catalog may hold any number

Storage

Parameter	Value	Notes
Store: a key	1 - 15 characters	from `!` to `~`, no spaces
Store: a value	1024 bytes	
Store: all values	4096 bytes	of one application; kept when it is removed, for when it comes back
Files: all of them	64 KB	what came in the package and what the application wrote, each file counted as the whole 4 KB blocks it takes on the device: 16 blocks. Every installed application has this much, whatever the others take
Files: a name	1 - 23 characters	`a–z 0–9 . - _`, not starting with a dot

Network

Parameter	Value	Notes
Transport	HTTP, HTTPS	made by the host; an application has no sockets
HTTPS trust	browsers' roots	a server with a certificate of its own making is refused; plain `http` works in a home network
Where its own requests may go	4 hosts	named in the manifest, or the server the user gave in a setting. A poll is not held to them: it goes where its `url` says
Polls in a manifest	4	each every 10 s to 1 day; on battery not closer than 300 s
Requests at once	1	of the whole device; 2 more may wait
A request's headers / body	1023 / 512 bytes	up to 4 header lines
An answer picked by the host	256 KB	JSON, up to 16 paths, a value up to 63 bytes
Streamed answers	4 MB	pieces of up to 1024 bytes; about 90 KB/s over plain HTTP
Redirects followed	3	for `GET`; a request of the application's own stays inside its hosts
An HTTPS request	about 6 s	most of it the handshake

Time and background work

Parameter	Value	Notes
Timers (on the screen)	4	one shot, in milliseconds; stop when the application leaves the screen
Alarms (on the screen or off it)	4	in seconds, at a moment, or repeating every 60 s to 1 day; a restart forgets them
Device events (intents)	3 kinds	started, time changed, network up

Parameter	Value	Notes
Notifications waiting	4	of the whole device
Taking the screen	once per 30 s	`request_focus()`; the host decides
Clock	unix seconds	from the network; 0 until first set. Local time and its zone are the host's

Text

Parameter	Value	Notes
Encoding	UTF-8	everywhere; lengths travel with pointers, nothing needs a terminator
Always drawn	printable ASCII, °	
Drawn with a glyph pack	3 packs	Latin with marks, Cyrillic, Greek: added by the user on the device's page
Without the pack	spelled in ASCII	"Moskva", "Strasse", "Athina"; what has no spelling is drawn as ?
One drawn text	127 bytes	longer is cut between characters
Sizes	6	capitals of 8 to 80 px; two typefaces, the user's choice
Languages of an application	27 known	English built in; others as tables the host loads (section 12)

3 Execution model

The host loads an application to call a handler and can unload it after the call.

Application lifecycle

What happened	The instance	Handlers, in order
The user opened the application	loaded, stays while it has the screen	`app_init, app_on_focus, app_on_draw ... app_on_blur, app_on_terminate`
An answer, an alarm, an intent or a change of settings, while it is on the screen	the same instance	`app_on_data / app_on_values / app_on_alarm / app_on_intent / app_on_settings`
The same, while it is not on the screen	loaded for the call, unloaded after it; what was on the screen is set aside and returns	`app_init, the handler, app_on_terminate`
It asked for the screen (`request_focus()`) and the host agreed	stays, on top of what was there	`... app_on_focus, app_on_draw`
It left the screen: BACK, the menu, another application on top	unloaded	`app_on_blur, app_on_terminate`

Handler guarantees

- Handlers run one at a time, each to completion. Locking is unnecessary.
- A handler has 3 seconds. A handler that runs longer, or traps (for example, on an invalid pointer), is stopped. The application is then terminated and the screen shows an error. The host keeps running.

- Handlers return `void`. A redraw, the screen or an exit is obtained with a request. A request takes effect after the handler returns.
- The host calls `app_init()` before any other handler of an instance.
- Button events arrive in order and are all delivered. Pending timers with the same number are merged into one. The pieces of one answer arrive in order, in consecutive calls.
- Background delivery of an answer can be interrupted. If the user presses a button, the host unloads the instance and drops the remaining pieces. The application receives `app_on_terminate()` only. The host repeats the poll after one minute.

State

Global variables live until the application is unloaded. Unloading happens every time the application leaves the screen and after every background call.

State is loaded in `app_init()` and saved in `app_on_terminate()`, the last call every instance receives. A value that must survive a power loss is saved immediately after it changes.

Timestamps that stay valid across restarts: `time_now()`, or `time_uptime_ms()` together with `time_boot_id()`. The boot id changes after every restart.

Navigation and refresh

Navigation is a stack. The menu is at the bottom. An application opened from the menu is placed on top. A notification, a question from the host or an application that requested the screen is placed on top of the current screen.

A short press of BACK removes the top entry. A long press of BACK clears the stack and opens the menu.

Only the top entry is loaded. An application lower in the stack is unloaded and restores its state when it returns to the top.

The host controls the panel. It compares each new frame with the frame on the panel and sends only the changed area. It postpones a refresh that was not caused by a button press if the previous one was too recent.

The panel always shows a complete frame. Refresh timing is decided by the host.

Part II. Development guide

4 A first application

Requirements

- **wasi-sdk** 34 or newer: clang for the wasm32 target. Location: the `WASI_SDK_PATH` environment variable.
- **Python 3**: the tools use the standard library only.
- A checkout of the **perinkpheral** repository: `sdk/` and `tools/`.
- A **device** on the same network, with a firmware that has the development tools enabled (the default). The device shows its address on the second page of About. The tools read the address from `PERINKPHERAL_HOST`.

Create, build, run

```
tools/devtool/devtool.py new my-app --title "My app"     # a folder with the SDK's sample in it
tools/devtool/devtool.py run my-app                      # build, upload, restart the device into it
tools/devtool/devtool.py serve my-app                    # a page with the live screen, buttons, log, Run
```

`run` builds the application, uploads it and restarts the device into it. Duration: about ten seconds. The firmware stays untouched.

`serve` starts a local web page with the live screen, the four buttons, the device log and a Run button. The arrow keys, Enter and Backspace act as the buttons.

`new` also creates a `CMakeLists.txt` in the application's folder. The folder opens as a project in any IDE that supports CMake, such as CLion or VS Code.

The IDE resolves the SDK headers and the generated `strings.h` and `pictures.h`. The project has three build targets: `my-app` (build), `my-app-run` (build and run on the device) and `my-app-serve` (the live page).

The IDE requires `WASI_SDK_PATH` and `PERINKPHERAL_HOST` in its CMake environment.

Sample application

```
#include "app_text.h"
#include "host_api.h"

#define MARGIN 16

static const char PRESSES_KEY[] = "presses";
#define PRESSES_KEY_LENGTH ((int32_t)sizeof PRESSES_KEY - 1)

static int32_t presses;

static void draw_line(int32_t line, const char *text)
{
    int32_t line_height = gfx_font_height(HOST_FONT_TEXT);
    gfx_text(MARGIN, MARGIN + line * line_height, HOST_FONT_TEXT, text, app_text_length(text));
}

void app_init(void)                /* the state comes from the store */
{
    kv_get(PRESSES_KEY, PRESSES_KEY_LENGTH, &presses, sizeof presses);
}

void app_on_terminate(void)        /* and goes back before the instance is unloaded */
{
    kv_set(PRESSES_KEY, PRESSES_KEY_LENGTH, &presses, sizeof presses);
}

void app_on_button(int32_t event)  /* a handler changes the state and asks for a redraw */
{
    switch (event) {
    case BTN_UP_PRESSED:
    case BTN_OK_PRESSED:
    case BTN_DOWN_PRESSED:
        presses++;
        request_redraw(HOST_REDRAW_NORMAL);
        break;
    }
}

void app_on_draw(void)             /* and this paints the state, all of it, on a white surface */
{
    char       counter_buffer[32];
    app_text_t counter;
    app_text_start(&counter, counter_buffer, sizeof counter_buffer);
    app_text_add(&counter, "Presses: ");
    app_text_add_number(&counter, presses);

    draw_line(0, "Hello from wasm");
    draw_line(2, counter_buffer);
}
```

BACK is handled by the host here: a short press of BACK closes an application that has no `app_on_back()` export.

Build report

```
tools/mkapp/mkapp.py my-app --out build/my-app

build/my-app/app.wasm: 811 bytes, 5 host calls
  module  0.8 KB of 48.0 KB
  code    0.5 KB; running it takes about 13.0 KB of the interpreter's 64.0 KB
  memory  9.1 KB of 64.0 KB: 8.0 KB of stack, 1.1 KB of data; 54.9 KB left for app_alloc()
```

Line	What it measures	What to do when it runs out
`module`	the file: code and initial data. Executed from flash	move pictures and tables into `files/`; draw less by hand, use the kit
`code`	the code alone, and an estimate of what the interpreter needs to run all of it. **This is the scarce one**: a byte of code costs four of memory	see "Memory" in section 13
`memory`	the 64 KB page: the stack, the static data, and what is left for the arena	large buffers into `app_alloc()` under a mark; read files in parts

The build also fails in these cases:

- the module imports a function that `host_api.h` does not declare;
- the code contains a floating-point operation (the report names it);
- the code calls a `sys_*` function and the manifest lacks `"system": true`;
- the manifest exceeds a limit;
- a translation does not match `en.json` (section 12).

5 The package

```
my-app/
  manifest.json      what the application is, fetches and asks the user for
  my-app.c           the code: any number of .c files
  strings/           optional: en.json, ru.json ... what it says, a language a file      (section 12)
  pictures/          optional: drawings - PNG, PBM, or rows of '#' and '.'               (section 6)
  files/             optional: data installed beside the module, read with file_read()
```

`mkapp` builds the package from this folder. It compiles the sources with the SDK library, generates `strings.h` from `strings/` and `pictures.h` from `pictures/`, and checks all limits. The package contains `app.wasm`, `manifest.json`, the files and one string table per language.

The manifest

```
{
  "name": "plex",
  "title": "Plex Now Playing",
  "version": "0.1.0",
  "abi": 2,
  "hosts": ["{{server}}"],
  "polls": [
    { "url": "{{server}}/status/sessions", "every": "{{every}}",
      "headers": { "Accept": "application/json", "X-Plex-Token": "{{token}}" },
      "pick": ["MediaContainer.Metadata[0].title", "MediaContainer.Metadata[0].grandparentTitle"] }
  ],
  "settings": [
    { "key": "server", "type": "url",    "title": "Server" },
    { "key": "token",  "type": "secret", "title": "Plex token" },
    { "key": "every",  "type": "choice", "title": "Refresh",
      "options": ["15 s", "1 min", "5 min"], "default": "1 min" }
  ],
  "intents": ["started"],
  "icon": ["..........", ".#########.", ".#.......#.", ".#..#....#.", ".#..###..#.", ".#..####.#.",
           ".#..###..#.", ".#..#....#.", ".#.......#.", ".#########.", ".........."]
}
```

Field	Rule	Meaning
`name`	1 - 13 of `a–z 0–9 – _`; the folder's name	the identity: the folder on the device, the key in a catalog, the name of the store. Changing it makes another application
`title`	1 - 23 characters	what the menu and the page call it; translatable
`version`	up to 15 bytes	the developer's own; the page offers Update when a catalog has another
`abi`	2	the host API it was built for; another is refused at install
`fullscreen`	true / false (default)	false: the host's header is above, the surface is 400 x 260
`icon`	11 rows of 11 of # and `.`	in the menu, on the page, in a catalog. Optional
`polls`	up to 4	what the host fetches on a schedule (below)
`hosts`	up to 4, each up to 63 bytes	where `http_request()` may go: a name, a name with a port, or `{{key}}` of a `url` or `link` setting. Only for requests of the code's own: an application that has polls alone needs none
`settings`	up to 8	the form on the device's page (below)
`intents`	`started`, `time_changed`, `network_up`	device events the application is loaded for; an unknown one fails the build
`system`	true / false (default)	asks for the `sys_*` calls; the page tells the user before installing
`companion`	an `https` address	a page of the developer's, opened from the settings form, for what a form cannot do

The host ignores unknown keys. `mkapp` validates the keys listed above.

Polls

Field	Rule	Meaning
`url`	up to 511 bytes; `http` or `https`, or beginning with `{{key}}` of a `url` setting, or exactly `{{key}}` of a `link` setting	`{{key}}` of any setting, anywhere in it, is replaced by the stored value encoded for a URL: every character but letters, digits and `– . _ ~` becomes `%XX`, **a slash too**. A setting is therefore one part of a path, never two: `owner/name` is two settings, `/repos/{{owner}}/{{name}}`. Only the `url` setting a URL begins with and the `link` that is the whole of it go in as they are. A poll that needs a setting which is not set fails with `HOST_FAILED_NOT_READY` - a header's too, so a token in a header makes the token required
`every`	10 to 86400 seconds, or `"{{key}}"` of a setting	with a setting, its value is the pace as a person writes it - a number and a unit, of which the first letter is read: `s`, `m`, `h`, `d` ("30 s", "15 min", "1 h", "1 day"); a bare number is seconds. The user's choice takes effect with no code
`headers`	up to 4; a value up to 255 bytes	`{{key}}` works in values: this is how a token travels
`pick`	up to 16 paths of up to 63 bytes	`current.temperature_2m`, `daily.code[0]`: the host reads the JSON and hands over the values. Without it the answer arrives in pieces

Polls are numbered from 0 in the order of the list. Handlers receive this number.

Settings

Settings are declared in the manifest. The device's web page builds the settings form from the manifest alone. Each value is stored as text in the application's store, under the key of the setting.

The application reads a setting with `kv_get()` and the key of the setting.

When the user saves the form, the host calls `app_on_settings()` and repeats all polls immediately.

Type	On the page	Stored value and notes
`text`	a line of text	as typed, up to 255 bytes
`number`	a number	digits, an optional sign and decimal point
`choice`	one of up to 8 `options`	the option itself, 1 - 23 characters. A translation gives options labels; the stored value does not change
`toggle`	a checkbox	`"1"` or `"0"`; nothing while never touched
`secret`	a password field	the page can write it and never reads it back. The application's own code reads it with `kv_get()` like any setting
`place`	a place found by name	the name; the page fills the two `number` settings named as its `latitude` and `longitude`
`url`	the user's own server	a scheme, a host and maybe a port, nothing after: `http://192.168.1.10:8123`
`link`	a whole address the user was given	`https://host/path?key`, up to 255 bytes; kept from the page like a secret. `webcal://` is taken as `https://`

- `default` is display-only: the page shows it while the setting is empty, and the store stays empty. The code applies the same default when `kv_get()` finds no value.
- `url` and `link` settings are read-only for the application: `kv_set()` on their keys fails. When the user changes such a setting, the host erases the secrets that were not entered again, so a token stays with the server it was entered for.
- `title` is 1 to 31 characters. `help` is up to 95 characters.

6 The screen

Surface and colour

- The origin is the top left corner of the surface. `gfx_width()` and `gfx_height()` return its size. Layout is based on these values.
- Drawing calls work inside `app_on_draw()`. The surface is white at the start of each call. In any other handler, `gfx_*` and `ui_*` calls return `HOST_ERR_NOT_DRAWING`. Drawing is clipped to the surface.
- `gfx_color()` sets the colour for the following calls, from `HOST_PAPER` (0) to `HOST_INK` (16). The colour is `HOST_INK` at the start of each `app_on_draw()`. Intermediate values are dithered greys. Greys are intended for filled areas only: thin lines and small text become unreadable.
- The standard left and right margin is `HOST_UI_MARGIN` (16 px).

Fonts

A font constant selects a size. The typeface is a user setting: Departure Mono (monospaced) or Jersey (proportional). Character widths differ between the two, so layout is based on measured widths.

Text measurement: `gfx_text_width()`, `gfx_font_height()`, `gfx_font_ascent()` and `gfx_font_cap_height()`.

Size	Capitals, px	Line, px	Characters in 400 px	For
`HOST_FONT_SMALL`	8 / 10	15 / 21	57 / 50	a unit, a caption beside something larger
`HOST_FONT_TEXT`	16 / 15	30 / 28	28 / 33	what is read; the host's own text
`HOST_FONT_MEDIUM`	24 / 25	45 / 43	19 / 21	a title, a value among others
`HOST_FONT_LARGE`	32 / 30	60 / 56	14 / 16	a notice's lines

Size	Capitals, px	Line, px	Characters in 400 px	For
`HOST_FONT_DISPLAY`	48 / 50	90 / 86	9 / 10	the one value of a screen
`HOST_FONT_HUGE`	80 / 80	150 / 140	5 / 6	a clock: five digits fill the width

Each cell gives the value for Departure Mono, then for Jersey.

- The `y` argument of `gfx_text()` is the top of the line. The baseline is at `y + ascent`; the top of the capitals is at `y + ascent − cap_height`.
- `HOST_FONT_TABULAR`, added to a font, gives all digits the same width. It is intended for numbers that change in place, such as a clock or a countdown. Every call that takes a font accepts the flag. Measurement requires the same flag as drawing.
- `gfx_text_fit()` shortens a text to a given width and appends "...". It cuts between characters, so the result is valid UTF-8.

Text beyond ASCII

All text is UTF-8. The host draws each character using the first available option:

1. The glyph of the character. Printable ASCII is always available. Latin letters with diacritics, Cyrillic and Greek are available when the user has installed the glyph pack (Settings, Language, on the device's web page).
2. An ASCII transliteration: "Moskva", "Strasse", "Athina". Typographic dashes and quotes become plain ones.
3. A question mark.

The degree sign is built in at every size.

The width of a text depends on which option was used. Text is therefore measured before it is placed.

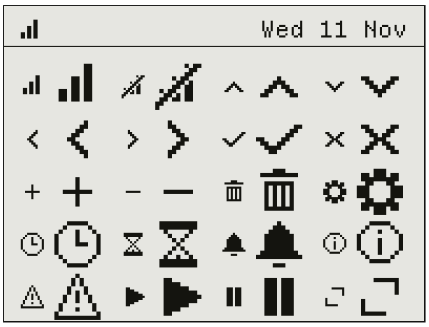
Limits on user-visible text, such as a title or an option, are given in characters. A field holds half as many two-byte characters as one-byte characters.

Pictures and icons

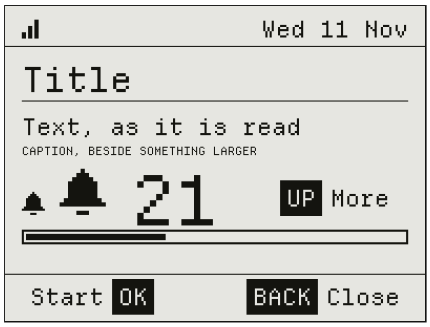
The host provides 20 built-in icons. Each icon is drawn on a grid of 11 x 11 cells. An application can use them on its own screens.

`gfx_icon()` draws an icon with a cell size of 1 to 16 pixels. `ui_icon()` draws an icon at one of three standard sizes: 22, 44 or 99 px.

Constant	Meaning	Constant	Meaning
`HOST_ICON_SIGNAL`	connected	`HOST_ICON_NO_SIGNAL`	not connected
`HOST_ICON_UP`	arrow up	`HOST_ICON_DOWN`	arrow down
`HOST_ICON_LEFT`	arrow left	`HOST_ICON_RIGHT`	arrow right
`HOST_ICON_CHECK`	selected, done	`HOST_ICON_CROSS`	cancel, close
`HOST_ICON_PLUS`	add	`HOST_ICON_MINUS`	remove
`HOST_ICON_TRASH`	delete	`HOST_ICON_GEAR`	settings
`HOST_ICON_CLOCK`	time	`HOST_ICON_HOURGLASS`	waiting, timer
`HOST_ICON_BELL`	notification, alarm	`HOST_ICON_INFO`	information
`HOST_ICON_WARNING`	warning, error	`HOST_ICON_PLAY`	start, play
`HOST_ICON_PAUSE`	pause	`HOST_ICON_UPDATE`	update; also the project's mark



Built-in icons at cell sizes of 2 and 4 pixels



Basic kit parts: title, text, caption, icons, value, button, bar, button row

An application's own pictures are placed in the `pictures/` folder. Supported formats are PNG, PBM and text. A text file contains rows of `#` (ink) and `.` (paper) and can hold several named pictures:

```
// pictures/things.txt
drop:
.....#.....
....###....
...#####...
..#######..
```

```
#include "pictures.h"   /* made by mkapp */

gfx_picture(x, y, PICTURE_DROP, 4);    /* 44 px: a cell 4 pixels */
gfx_picture(x, y, PICTURE_DROP, 2);    /* 22 px, the same drawing */
```

Rules for text files:

- A line of the form `name:` starts a picture.
- Rows before the first such line form a picture named after the file.
- All rows of a picture have the same length.
- A picture is at most 255 cells wide and 255 cells high.

The constant is `PICTURE_` followed by the picture name in upper case. Characters other than letters and digits are replaced with `_`. Example: `half-moon` becomes `PICTURE_HALF_MOON`.

`mkpicture.py --show` prints the pictures as the tool read them.

A picture is stored as one bit per cell. `gfx_picture()` draws it with a cell size of 1 to 16 pixels, so one picture can be used at several sizes.

The recommended grids are those of the built-in icons: 11 cells, or 22 and 44 cells for more detail. Pictures on these grids match the icons in weight.

A picture that is too large for the module belongs in `files/` and is drawn with `gfx_bitmap_file()`. The host reads the file directly; the picture does not use the application's memory.

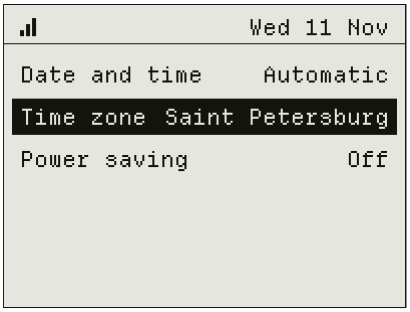
The kit

The kit is the set of interface parts that the host's own screens are built from. Applications can use the same parts through the `ui_*` calls.

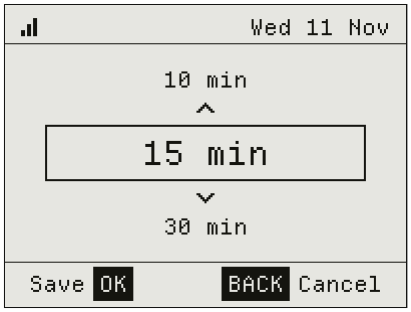
Using the kit is optional: `gfx_*` calls can draw anything. The kit has three advantages. The application looks like the rest of the system. It follows future changes of the look without a rebuild. It is smaller, because the drawing code is in the host.

Part	What it is	Use it for
`ui_list_row, ui_list_first, ui_list_visible`	the host's list: a row a line, a value or an icon at the right edge, the chosen row white on a black bar, a scroll bar. With nothing chosen it is a list of facts that UP and DOWN move	anything with more than three items; a menu of the application's own; settings
`ui_cell, ui_cell_fit`	a name, a picture and a value in a box of any size, the value as large as fits	a dashboard: one, two or four values at a glance
`ui_value`	a value and its unit after it on the baseline: "18 km/h"	every number that has a unit

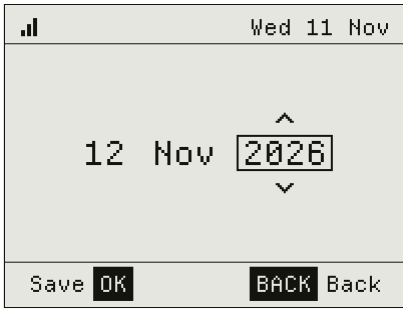
Part	What it is	Use it for
`ui_notice`	two large lines, a line of text, a large icon, what the buttons do	a question, an error, "nothing to show", work going on
`ui_reason`	the notice for what a request came to: "Loading", "Not set up", "No network", "Access denied", "No answer"	every screen that waits for the network
`ui_actions, ui_button`	the row along the bottom: what OK does from the left, what BACK does from the right	every screen where a button does something that is not obvious
`ui_picker`	one of a row of values, the chosen one framed, its neighbours above and below	choosing among a few values with UP and DOWN
`ui_date`	a time, a date or both, set a part at a time	any moment the user has to give
`ui_frame`	a frame with an arrow for each way UP and DOWN can go	marking the one value on a screen that the buttons change
`ui_bar`	how much of how much, filled from the left	a quantity. It changes with the data; it is not an animation
`ui_title, ui_text, ui_text_lines, ui_icon`	a page's title with its line; text in the host's four styles, on one line or broken over several; an icon at its three sizes	the plain parts of a page



ui_list_row: a list of rows with values



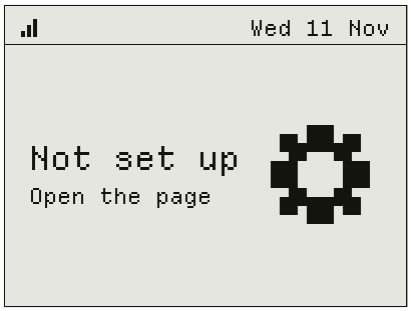
ui_picker: selecting one value



ui_date: setting a date or a time



ui_notice: a question



ui_notice: a message with a reason



ui_notice with is_working: work in progress

Button handling for lists, pickers and dates is implemented in the SDK library (`app_ui.h`). The application calls it from `app_on_button()` and acts on the result (section 7).

The `kit` application in the catalog shows every part on a separate page. Its source code is a working example for each part.

Requesting a refresh

request_redraw(...)	What the host does	For
`HOST_REDRAW_NORMAL`	from a button handler: at once. From a timer, an alarm or data: when the user's policy allows, by default not more than once a minute	nearly everything
`HOST_REDRAW_SOON`	now, although nobody pressed anything	news the user is waiting for: the answer to a refresh they asked for

request_redraw(...)	What the host does	For
`HOST_REDRAW_CLEAN`	a full refresh: 2 - 3 s, a blink, no traces	a picture that must be clean; the end of a row of quick ones
`HOST_REDRAW_NO_BLINK`	now, and never as the full refresh the host makes after a few partial ones	stepping through a list; a value that changes every second

Redraw requests are unlimited.

Each refresh uses battery energy and wears the panel. Frequent changes also make the device distracting. Guidelines: section 13.

7 Buttons

The buttons are, in order: UP, OK, DOWN, BACK. The host uses them the same way on every screen. Applications are expected to keep these meanings.

Each press produces two kinds of events. The raw events are `KEYDOWN` and `KEYUP`. The interpreted events are `PRESSED`, sent when a short press ends, and `LONG_PRESSED`, sent after 600 ms while the button is still held. A long press does not produce `PRESSED`.

Most applications handle only `PRESSED` and `LONG_PRESSED`.

The event value combines the button and the action. Each combination has a constant:

```
void app_on_button(int32_t event)
{
    switch (event) {
    case BTN_OK_PRESSED:       start();  break;
    case BTN_OK_LONG_PRESSED:  reset();  break;
    }
    request_redraw(HOST_REDRAW_NORMAL);
}
```

BACK

The application	A short press of BACK
does not export `app_on_back()`	leaves it: one step down the stack. Nothing to write
exports `app_on_back()`	is delivered there. Go a page back; from the first page call `request_exit()`

A short press of BACK goes to `app_on_back()`, or closes the application if it has no such export. A long press of BACK opens the host's menu. `app_on_button()` receives all other events.

`request_exit()` removes the application from the top of the stack. The user returns to the previous screen. This also applies to an application that took the screen with `request_focus()`.

Lists, pickers, dates

```
#include "app_ui.h"

static app_list_t list = { .count = 12 };          /* .selected = -1: facts, nothing to choose */

void app_on_button(int32_t event)
{
    switch (app_list_button(&list, event)) {
    case APP_UI_DONE:      open_item(list.selected);               break;   /* OK */
    case APP_UI_CHANGED:   request_redraw(HOST_REDRAW_NO_BLINK);   break;   /* UP, DOWN */
    default:               break;
    }
}

void app_on_draw(void)
{
    int32_t first = app_list_first(&list);
    for (int32_t row = first; row < first + ui_list_visible(&list) && row < list.count; row++) {
        ui_list_row(&list, row, label_of(row), label_length(row), "", 0, HOST_ICON_NONE);
    }
}
```

`app_picker_t` and `app_date_t` follow the same pattern. The application keeps a struct, passes button events to one function and draws with another. The button function returns *changed*, *done*, *cancelled* or *ignored*.

8 Storage

The store

`kv_get()` and `kv_set()` read and write small values by key. Each application has a private namespace.

The store holds the application's state and its settings. The data is kept when the application is removed and is available again after a reinstall.

```
typedef struct { int32_t version; int32_t minutes; int64_t ends_at; } state_t;
static state_t state;

void app_init(void)
{
    if (kv_get("state", 5, &state, sizeof state) != sizeof state || state.version != 2) {
        state = (state_t){ .version = 2, .minutes = 25 };     /* nothing stored, or an older layout */
    }
}
```

- A version field in every stored struct, checked on load, is required for upgrades: a newer version of the application reads data written by an older one.
- Every write wears the flash memory. A write is justified only when the data has changed, not on every timer tick.
- `kv_set()` with a length of 0 deletes the key.

Files

Files are intended for data that does not fit in a store value: a downloaded list, a cache, a picture.

`file_read()` and `file_write()` take an offset. A file can be read and extended in parts, so it never has to be held in memory in full.

The files from the `files/` folder of the package are installed with the application.

9 The network

All network access goes through the host. The application describes a request; the host performs it and delivers the answer. The host performs one request at a time for the whole device. Delivery is independent of whether the application is on the screen.

Network availability is reported in the answer: a request made without a network fails with `HOST_FAILED_NO_NETWORK`.

Way	When	The answer arrives as
a poll with `pick`	on a schedule, with no code running	`app_on_values()`: the values of the paths, once
a poll without	on a schedule	`app_on_data()` in pieces, under the poll's number
`http_request()`	when the code decides: after a press, as a second step	`app_on_data()` in pieces, under the tag the application chose (16 - 255)

Values extracted by the host

```
#include "app_values.h"
enum { VALUE_TEMPERATURE, VALUE_CODE };            /* the order of "pick" in the manifest */

void app_on_values(int32_t request, int32_t http_status)
{
    int32_t tenths, code;
    last_status = http_status;                     /* for ui_reason(), when there is nothing to show */
    if (http_status == 200 &&
        app_value_number(VALUE_TEMPERATURE, 1, &tenths) &&      /* "21.46" becomes 215 */
        app_value_number(VALUE_CODE, 0, &code)) {
        keep(tenths, code);                        /* into the store */
    }                                              /* else: what was shown before stays */
    request_redraw(HOST_REDRAW_NORMAL);
}
```

Numbers are integers. `app_value_number()` returns the value scaled by the requested number of decimals, rounded.

A path that is absent from the answer has no value: `app_value_number()` returns false. An absent value means no data.

Streamed answers

```
void app_on_data(int32_t request, const uint8_t *data, int32_t length, int32_t flags, int32_t http_status)
{
    if (flags & HOST_DATA_FAILED) {                /* alone, with no data: http_status is a HOST_FAILED_* */
        last_status = http_status;
        return;
    }
    if (flags & HOST_DATA_FIRST) {                 /* the HTTP status comes with the first piece */
        last_status = http_status;
        begin_reading();                           /* always start afresh here */
    }
    read_piece(data, length);                      /* valid only until the handler returns */
    if (flags & HOST_DATA_LAST) {
        take_what_was_read();                      /* only now is the answer whole */
        request_redraw(HOST_REDRAW_NORMAL);
    }
}
```

- Each piece is processed when it arrives; only the data to display is kept. Suitable parsers: a state machine, the SDK's `json_pick`, or a custom one. An answer can be several megabytes; the application has 64 KB.
- The displayed data is replaced only on `HOST_DATA_LAST`. Delivery can stop without a last piece and without a failure (section 3). `HOST_DATA_FIRST` always starts a new answer.
- Inside the handler, `data_response()` returns the response metadata: status, Content-Length, Content-Type, ETag, Last-Modified, Location, Link and Retry-After.

Requests from code

```
enum { REQUEST_TOGGLE = HOST_REQUEST_FIRST };
static const char URL[]     = "{{server}}/api/services/light/toggle";
static const char HEADERS[] = "Authorization: Bearer {{token}}\nContent-Type: application/json";

http_request(REQUEST_TOGGLE, HOST_HTTP_POST, URL, sizeof URL - 1, HEADERS, sizeof HEADERS - 1,
             body, body_length);
```

- Requests are allowed only to the hosts listed in the manifest. `{{key}}` placeholders work in the URL and in header values, so the code does not need to read the token.
- `HOST_ERR_NO_SPACE` means that the request queue of the device is full (two waiting requests). The request can be repeated later.
- The host follows up to three redirects for a `GET`. A redirect to a host outside the list fails with `HOST_FAILED_NOT_ALLOWED`. A request with a body is sent once; its redirect response is delivered as the answer.
- `poll_now(n)` runs poll `n` immediately, for example when the user presses a refresh button. `poll_set_interval(n, seconds)` changes the interval of a poll from code, for example to honour a Retry-After header.

Failed requests

Status	What happened
`HOST_FAILED_NO_NETWORK (-1)`	the device is not on a network
`HOST_FAILED_NO_ANSWER (-2)`	the server did not answer, or not to the end
`HOST_FAILED_NOT_READY (-3)`	a `{{setting}}` the request needs is not set
`HOST_FAILED_NOT_ALLOWED (-4)`	redirected to a host the manifest does not list
`HOST_FAILED_TIMEOUT (-5)`	the server was reached and did not answer in time
`HOST_FAILED_NOT_TRUSTED (-6)`	https, and the certificate is not one the device trusts
`HOST_FAILED_TOO_LARGE (-7)`	over 256 KB to pick apart, or over 4 MB in pieces
`HOST_FAILED_NO_MEMORY (-8)`	the device has no room for the request just now

The last status is kept in the store. When there is no data to display, `ui_reason()` takes the status, draws the standard notice for it and returns whether it drew anything:

Status	The notice	Returns
`0 - nothing was asked yet`	Loading	1
`HOST_FAILED_*`	Not set up, No network, No answer ... one for each	1
`401, 403`	Access denied / Check the settings	1
`404`	Not found / Check the address	1
`any other that is not 2xx`	No answer / Code 500	1
`200 - 299`	nothing is drawn: an answer came, and what is missing in it is the application's to say	0

10 Background work

The host loads an off-screen application for a single call when one of these events occurs:

To	Use	Notes
take data on a schedule	a poll in the manifest	needs nothing after a restart: the manifest is the schedule
act at a moment	`alarm_set, alarm_set_at, alarm_set_every`	a restart forgets alarms: set them again on the `started` intent, from a moment in the store
tell the user something	`notify`	the host shows it over whatever is there until it is dismissed, and gives the screen back
show itself	`request_focus`	the application becomes the screen, if the host agrees: once in 30 s at most
keep a line in sight	`header_status`	an icon and a few words in the header above every screen, while the application is away
act when the device starts, joins a network, or its clock is set	an intent in the manifest	`app_on_intent()`

Notifications

```
void app_on_alarm(int32_t id)                    /* loaded for this one call */
{
    host_notice_t notice = { .first = "Time", .second = "is up", .icon = HOST_ICON_BELL };
    notify(&notice);
}
```

After `notify()` returns, the host manages the notification. It shows the notice under the header, keeps it until the user presses OK or BACK, and then restores the previous screen.

The notification is managed entirely by the host. The application stays unloaded while it is displayed.

`notify()` is preferred over `request_focus()` when the message fits in two lines.

Alarms, timers

Call	Fires
`timer_set(id, milliseconds)`	once, `app_on_timer(id)`, only while the application is on the screen. Set it again in `app_on_focus()`
`alarm_set(id, seconds)`	once, `app_on_alarm(id)`, on the screen or off it. Counts time, whatever the clock is set to
`alarm_set_at(id, unix_time)`	once, at a moment of the clock; follows the clock when it is set
`alarm_set_every(id, seconds, first_in)`	again and again, every 60 s to a day. Every call loads the application: use the longest period that will do

11 Time

- `time_now()` returns Unix time in seconds. It returns 0 until the device has obtained the time. This case requires handling in the application.
- Displayed time is local time. The host owns the time zone. `time_local()` returns the current local date and time, `time_local_at()` converts any moment, and `time_from_local()` converts a local date and time back to Unix time, for example for `alarm_set_at()`.

- `time_text()` formats a moment as text in the device's language. Applications do not need their own names of days and months. Formats: "15:04", "Fri 13 Nov", "Friday", "13 November", and a relative form: "15:04", "Tmrw 15:04", "Sat 15:04", "21 Nov".
- The `time_changed` intent delivers a call when the clock is set or changed.

12 Languages

An application displays its text in the language of the device. The developer provides the translations. The host loads the translation for the current language.

```
strings/en.json    { "no_events": "No events", "starts_in": "In {0} min" }
strings/ru.json    { "no_events": "Нет событий", "starts_in": "Через {0} мин",
                     "manifest": { "title": "Календарь",
                                   "settings": { "every": { "title": "Обновление",
                                                            "options": { "15 min": "15 мин" } } } } }
```

```
#include "strings.h"                              /* made by mkapp: STR_NO_EVENTS, STR_STARTS_IN, app_string() */

const char *text = app_string(STR_NO_EVENTS);     /* the text in the device's language: no call, no copy */

const host_text_argument_t minutes = { .number = 10 };
char line[32];
int32_t length = text_fill(app_string(STR_STARTS_IN), app_text_length(app_string(STR_STARTS_IN)),
                           &minutes, 1, line, sizeof line);          /* "In 10 min" */
```

- `en.json` is required. English is compiled into the module and is used when a translation is unavailable. Every other file contains exactly the same keys. The key `manifest` is allowed in translations only (see below).
- When the host loads the application, it copies the string table of the device's language into the application's memory, over the English table. `app_string()` then returns the translated text with no extra call.
- `text_fill()` replaces `{0}`, `{1}` in a string with numbers or texts. A translation can place them in any order. A string has one form for all numbers, for example "min".
- The `manifest` key of a translation contains the translated parts of the manifest: the title, the title and help of each setting, and labels for the options of a choice. The stored option value does not change; only its label is translated.
- The build fails if a language is not listed in `sdk/languages.json` (27 languages, two-letter codes), if a translation has a missing or an extra key, or if a string contains a character that the glyph packs of the language do not cover.
- Translations are usually about a third longer than English. The kit shortens text that does not fit with "...". Translated screens require a check on the device.

13 Design rules

These rules apply to all applications. The host's own screens follow them. Most are conventions; the host checks only a few.

Screen content

One main value per screen.
The device is read from about a metre, in about a second. The main value is the largest element; everything else is small.

Static screens.
A screen changes only when its data changes. A bar shows a quantity. Work in progress is indicated by a notice with `is_working`.

Redraws only on content change.
A timer requests a redraw only if the picture differs. A clock redraws once a minute, on the minute. By default the host allows one refresh per minute that was not caused by a button press; a design is expected to fit this rate.

Black on white.
Content is ink on paper. Grey is for filled areas only. White text on a black bar is reserved for the selected item and for new information.

Short, standard interface text.
Labels use common words: "Save", "Remove", "Not set up". A label that needs an explanation is the wrong label.

Memory

Code size before data size.
Each byte of executed code uses about four bytes of the interpreter's 64 KB. Each byte of data uses one byte of the 64 KB page, which is mostly free. Preferred: a table over a chain of `if` statements, the kit over custom drawing, `pick` over a custom parser, `time_text()` over custom date formatting.

Shared helpers stay out of line.
A small helper called from many places is marked `__attribute__((noinline))`. The build report shows the effect.

Pictures are stored as bits.
Pictures belong in `pictures/`. Large pictures belong in `files/`.

Answers are processed as a stream.
An answer is processed as it arrives; only the data to display is kept. Working buffers come from `app_alloc()` and are released with a mark.

State and time

State is saved before every unload.
All state that matters is in the store when `app_on_terminate()` returns. Test: open the application, leave it, open it again; the screen looks the same.

Displayed data stays until new data is complete.
After a failed or interrupted request the screen keeps the previous data.

An empty screen has an explanation.
`ui_reason()` shows "Not set up", "No network" or "Loading".

The clock can be unknown and can change.
`time_now() == 0` is a valid state. `alarm_set()` is for a delay ("in ten minutes"); `alarm_set_at()` is for a clock time ("at seven").

Interaction and data

Text is entered on the web page.
Text such as a token, an address or a city is a manifest setting, entered on the device's web page. On the device, a value is chosen from a few options with UP and DOWN.

The screen is taken rarely.
Notifications are for events the user asked to be told about, such as a timer or a meeting. Other information waits until the user looks.

User data goes only where the user directs.
The hosts list contains only the hosts the application needs. The user's own server is a `url` setting. A token is a `secret` setting, sent through a header placeholder by the host.

Polling is no more frequent than data changes.
The interval is a setting with a reasonable default. On battery power the host limits every poll to one per five minutes.

14 Tools and debugging

Command	What it does
`tools/mkapp/mkapp.py <dir> --out <dir>`	builds one application: compiles, links the SDK's library, checks, measures
`tools/devtool/devtool.py new <dir> [--title T]`	a folder to start from: the SDK's sample under that name
`tools/devtool/devtool.py run <dir> [--shot f.png]`	builds, uploads, restarts the device into the application
`tools/devtool/devtool.py serve [dir]`	a local page: the live screen, the buttons, the log, Run
`tools/devtool/devtool.py shot [f.png]`	the frame the panel shows, as a picture
`tools/devtool/devtool.py press 3 3 2L [--shot f.png]`	presses: 1 - 4 short, 1L - 4L long, in the device's order
`tools/devtool/devtool.py open <name>`	puts an installed application on the screen
`tools/devtool/devtool.py log [--follow] [--app]`	the device's log; `--app` keeps the lines of applications
`tools/devtool/devtool.py set <app> <key> <value>`	a value into an application's store: a setting, without the page
`tools/devtool/devtool.py status`	firmware, network, free memory, why the device last started
`tools/mkpicture/mkpicture.py <files> --show`	shows drawings as rows of # and . : what `mkapp` will make of `pictures/`
`tools/mkvideo/mkvideo.py <video> --out <bin>`	a video into a 1-bit stream small enough to be a file of an application
`tools/catalog/mkcatalog.py <folder> --out <dir>`	builds a folder of applications into a catalog

`host_log(level, text, length)` writes one line of up to 200 characters to the device log. It replaces `printf`.

`press ... --shot` waits for the panel to finish and saves the frame. This can be used to script regression tests of the application's screens.

Common problems

Symptom	Most likely
Nothing on the screen	drawing outside `app_on_draw()`, or no `request_redraw()` after the state changed
The state is gone after leaving and returning	it lived in globals. Store it in `app_on_terminate()`, read it in `app_init()`
"<title> crashed" with "memory alloca..."	more code ran than the interpreter has room for. The build warned: see "Memory" in section 13
"<title> cannot start", "Update the app."	built against another `host_api.h` than the device's firmware. Rebuild and install again
A poll answers -3	a `{{setting}}` of its URL or headers is not set
https answers -6	a certificate of the server's own making, or expired. Use `http` inside a home network
`http_request()` returns -4	both waiting places of the device are taken; ask again from the next handler
Grey shadows of earlier pictures	a row of `NO_BLINK` redraws that never ended with a `CLEAN` one
Letters as ? or in Latin spelling	the glyph pack for the alphabet is not installed on the device; or the text was cut by bytes
A press does nothing right after the device starts	an answer is being taken in the background; the press ended it and the screen is being drawn again

15 Publishing

A device installs applications from *catalogs*. A catalog is a folder of static files with an index. It can be hosted anywhere that serves HTTPS, for example as raw files of a git repository or as a static site.

```
tools/catalog/mkcatalog.py my-apps --out catalog       # builds every application of the folder
                                                       # catalog/catalog.json and a folder for each
```

- The index lists the name, title, version and icon of each application, and its files with their sizes and SHA-256 hashes. The device accepts a file only if both the size and the hash match.
- Trust model: HTTPS to a source that the user selected, plus the sandbox. Packages are unsigned.
- The official catalog is the `perinkpheral-apps` repository. It is the first source on every device. The user can add up to three more sources on the device's web page by entering a repository address.
- The web page shows a summary of each installed application, taken from its manifest: the servers it connects to, the polling interval, what it stores, and whether it starts with the device.
- `version` increases with every change. The device compares versions to detect updates.

Part III. Reference

This part lists every declaration of `sdk/host_api.h` for ABI 2.

Numeric arguments and return values are `int32_t` unless stated otherwise. A text argument is a pointer and a length in bytes; the text is UTF-8.

A call that returns no value of its own returns `HOST_OK` or a negative `HOST_ERR_*` code.

16 Return codes

Code	Meaning
`HOST_OK (0)`	done
`HOST_ERR_NOT_DRAWING (-1)`	a drawing call outside `app_on_draw()`
`HOST_ERR_ARGUMENT (-2)`	a pointer outside the application's memory; an unknown font, icon, timer, style; a size beyond reason
`HOST_ERR_NOT_FOUND (-3)`	nothing stored under the key; no such file; no time yet; a path the answer did not have
`HOST_ERR_NO_SPACE (-4)`	the value is larger than the buffer; the store or the files are full; all waiting requests or notifications are taken
`HOST_ERR_FAILED (-5)`	the storage itself failed

17 Exports

All exports are optional. The host skips a handler that the module does not export.

Export	Called
`void app_init(void)`	once, right after the instance is loaded and its strings are in place
`void app_on_focus(void)`	the application got the screen. Set `timer_set()` timers here
`void app_on_blur(void)`	it is about to lose the screen
`void app_on_terminate(void)`	the last call of an instance, whatever the reason: it left the screen, a call in the background is done, the device restarts, the application is removed. `app_on_blur()` comes first if it was on the screen
`void app_on_draw(void)`	the frame is wanted: paint all of it from the state. The surface is white
`void app_on_button(int32_t event)`	a button: `BTN_<UP|OK|DOWN|BACK>_<KEYDOWN|KEYUP|PRESSED|LONG_PRESSED>`. Never `BTN_BACK_PRESSED` or `BTN_BACK_LONG_PRESSED`
`void app_on_back(void)`	a short press of BACK, for an application with pages of its own
`void app_on_timer(int32_t id)`	a timer ran out

Export	Called
`void app_on_alarm(int32_t id)`	an alarm came, on the screen or off it
`void app_on_data(int32_t request, const uint8_t *data, int32_t length, int32_t flags, int32_t http_status)`	a piece of an answer: to poll number `request`, or to the request under that tag. `flags`: `HOST_DATA_FIRST`, `_LAST`, `_FAILED`
`void app_on_values(int32_t request, int32_t http_status)`	the answer to a poll with `pick`; read the values inside the handler
`void app_on_settings(void)`	the user changed the application's settings on the page
`void app_on_intent(int32_t intent)`	a device event the manifest asked for: `HOST_INTENT_TIME_CHANGED` (1), `_NETWORK_UP` (2), `_STARTED` (3)
`uint8_t *app_data_buffer(void)`	where the host puts a piece of data: 1 KB. Provided by `sdk/lib`
`uint8_t *app_strings_buffer(void)`	where the host puts the strings of the device's language. Made by `mkapp`

18 Drawing

Call	What it does
`gfx_width(void), gfx_height(void)`	the surface: 400 x 260 under the header, 400 x 300 full-screen
`gfx_color(color)`	how dark the calls after it draw: `HOST_PAPER` (0) to `HOST_INK` (16)
`gfx_text(x, y, font, text, length)`	text with the top of its line at y; returns the width drawn. At most 127 bytes
`gfx_text_width(font, text, length)`	the width it would take
`gfx_text_fit(font, text, length, max_width, fitted, capacity)`	the text cut with "..." to fit the width; returns its length
`gfx_font_height(font), gfx_font_ascent(font), gfx_font_cap_height(font)`	the whole line; from its top to the baseline; capitals and digits above the baseline
`gfx_rect(x, y, w, h, fill)`	`HOST_RECT_OUTLINE` or `HOST_RECT_FILLED`
`gfx_line(x0, y0, x1, y1)`	a line one pixel wide, both ends included
`gfx_picture(x, y, picture, cell)`	a picture as `mkapp` makes of `pictures/`: a byte of width, a byte of height, a bit a cell; a cell 1 - 16 px
`gfx_icon(icon, x, y, cell)`	one of the host's `HOST_ICON_*`, 11 cells square, a cell 1 - 16 px
`gfx_bitmap(x, y, w, h, bits)`	a bitmap pixel for pixel: a set bit takes the colour, a clear one leaves what is there
`gfx_bitmap_file(x, y, w, h, name, name_length, offset)`	the same, read by the host from a file of the application's at an offset
`gfx_qr(x, y, cell, text, length)`	a QR code of up to 200 bytes; returns its modules a side, and with a cell of 0 only that
`void request_redraw(flags)`	the picture is out of date: `HOST_REDRAW_NORMAL`, `_SOON`, `_CLEAN`, `_NO_BLINK`
`header_text(side, text, length)`	not a drawing call: from any handler, whenever the text changes. A line of the application's own in the host's header above its screen: `HOST_HEADER_LEFT` or `_RIGHT`, up to 63 bytes. A length of 0 gives the side back

19 The kit

Call	What it does
`ui_text(x, y, style, text, length, max_width)`	text in a style: `HOST_UI_TEXT`, `_TITLE`, `_CAPTION`, `_VALUE`; cut at `max_width` (0: not cut). Returns the width
`ui_text_lines(x, y, style, text, length, width, max_lines)`	the same over several lines: broken at its spaces to fit `width`, at most `max_lines` (1 - 8), the last cut with "..." when there is more. Returns the lines drawn
`ui_text_width(style, text, length)`, `ui_text_height(style)`	to centre and to stack
`ui_title(text, length)`	a page's title with its line; returns the y where the content begins
`ui_icon(x, y, icon, size)`	a host's icon at `HOST_UI_ICON_SMALL` (22 px), `_MEDIUM` (44), `_LARGE` (99)
`ui_actions(ok, ok_length, back, back_length)`	the row along the bottom, 44 px (`HOST_UI_ACTIONS_HEIGHT`). A length of 0: that button does nothing here, and the other stands alone in the middle
`ui_button(x, y, name, name_length, action, action_length, mirrored)`	one button's name on its plate with what it does beside it; returns the width
`ui_list_visible(list)`, `ui_list_first(list)`	of a `host_list_t` - count, selected (or -1), top, height (0: to the bottom), first: the rows in sight, and the first of them
`ui_list_row(list, index, label, label_length, value, value_length, icon)`	one row; a row out of sight draws nothing. Label and value never meet: a value of up to half the row stays whole and the label is cut with "..."
`ui_notice(notice)`	a `host_notice_t` on the whole surface: first[64], second[64], detail[96], ok[32], back[32], icon, is_working. Empty texts are not drawn
`ui_reason(status)`	the notice for a request's status. Returns 1 when it drew, 0 for an answer that came (2xx)
`ui_value(x, y, font, value, value_length, unit, unit_length)`	a value and its unit on the baseline; returns the width of both
`ui_value_width(font, value, value_length, unit, unit_length)`	the same, measured
`ui_cell(cell, picture)`	a `host_cell_t` - x, y, width, height, name[48], value[48], size - with a picture or 0. A cell that is the whole surface has its picture 88 px square, any other 44
`ui_cell_fit(cell, picture)`	the size the cell would have, 1 the largest. Give every cell of a screen the largest number among them
`ui_bar(x, y, width, value, maximum)`	how much of how much, 14 px high
`ui_frame(x, y, width, height, arrows)`	a frame, with `HOST_UI_ARROW_UP` and `_DOWN` for the ways there are to go
`ui_picker(picker)`	a `host_picker_t`: above[64], chosen[64], below[64], ok[32], back[32]. Draws the row of buttons too
`ui_date(date)`	a `host_date_t`: year, month, day, hour, minute; has (`HOST_DATE_HAS_TIME | _DATE`); selected (`HOST_DATE_HOUR` .. `_YEAR`, or `_NONE`); ok[32], back[32]

20 Navigation, storage and network

Call	What it does
`void request_exit(void)`	done: one step down the stack. From a button, back or timer handler
`void request_focus(void)`	from a handler off the screen: worth showing now. The host decides; once per 30 s at most
`notify(notice)`	a `host_notice_t` for the host to show until dismissed: first (required), second, detail, icon. `HOST_ERR_NO_SPACE` when four wait
`header_status(side, icon, text, length)`	an icon and up to 31 bytes in the header above every screen while the application is away. Both empty gives the side back; a restart clears it
`timer_set(id, milliseconds)`	one shot, id 0 - 3; 0 cancels; setting it again restarts it
`alarm_set(id, seconds)`	id 0 - 3; 0 takes it back
`alarm_set_at(id, int64_t unix_time)`	at a moment of the clock
`alarm_set_every(id, seconds, first_in)`	every 60 - 86400 s; the first after `first_in` seconds (0: a whole period)
`kv_get(key, key_length, value, capacity)`	returns the value's length. `HOST_ERR_NOT_FOUND`; `HOST_ERR_NO_SPACE` for a buffer too small
`kv_set(key, key_length, value, length)`	a length of 0 removes the key. Refused for a `url` or `link` setting
`file_size(name, name_length)`	the file's length, or `HOST_ERR_NOT_FOUND`
`file_read(name, name_length, offset, bytes, capacity)`	returns how many bytes were read: fewer at the end, 0 past it
`file_write(name, name_length, offset, bytes, length)`	inside the file or right at its end. A length of 0 at offset 0 removes it
`poll_now(poll)`	fetch a poll of the manifest now
`poll_set_interval(poll, seconds)`	another pace for it, 10 s to a day, kept through a restart
`http_request(tag, method, url, url_length, headers, headers_length, body, body_length)`	`HOST_HTTP_GET`, `_POST`, `_PUT`, `_DELETE`, `_PATCH`, `_HEAD`, `_PROPFIND`, `_REPORT`; headers as "Name: value" lines
`data_value(index, text, capacity)`	inside `app_on_values()`: the value of path `index` as text, not terminated; returns its length
`data_response(response)`	inside a data handler, a `host_response_t`: status, length, retry_after, content_type[48], etag[64], last_modified[32], location[128], link[192]
`text_fill(pattern, pattern_length, arguments, count, text, capacity)`	the pattern with `{0}` .. `{3}` replaced by `host_text_argument_t` - number, or text and length; returns the length written
`void host_log(level, text, length)`	`HOST_LOG_ERROR`, `_WARN`, `_INFO`, `_DEBUG`; one line of up to 200 characters

21 Time

Call	What it does
`int64_t time_now(void)`	unix seconds; 0 while the time is not known
`time_zone_offset(void)`	seconds to add for the user's local time, daylight saving included
`time_local(parts), time_local_at(int64_t unix_time, parts)`	a `host_local_time_t`: year, month 1 - 12, day, weekday 0 = Sunday, hour, minute, second
`int64_t time_from_local(parts)`	the moment at which the user's clock shows this; a day past the end of its month counts on into the next

Call	What it does
`time_text(int64_t unix_time, style, text, capacity)`	`HOST_TIME_CLOCK`, `_DATE`, `_WEEKDAY`, `_LONG_DATE`, `_NEAR`, `_NEAR_DAY`; in the device's language; returns the length
`int64_t time_uptime_ms(void)`	milliseconds since the device started; counts through sleep
`uint32_t time_boot_id(void)`	another number after every restart, never 0: keep it beside a stored uptime

22 The system API

The system API is the set of `sys_*` calls. The host's own pages use it.

Any application can use the system API if its manifest contains `"system": true`. The web page warns the user before installing such an application. A module that imports a `sys_*` call without this declaration is rejected.

Call	What it does
`sys_info(what, text, capacity)`	about the device, as text: `SYS_INFO_VERSION`, `_NAME`, `_NETWORK`, `_ADDRESS`, `_WIFI` ("online", "connecting", "setup"), `_SLOT`, `_FREE_MEMORY`, `_TIME_ZONE`, `_JOIN_CODE`
`sys_number_get(what, number)`, `sys_number_set(what, value)`	a setting of the host that is a number, with its range: `SYS_NUMBER_TIME_AUTOMATIC`, `_REFRESH_FULL_EVERY`, `_REFRESH_MIN_SECONDS`, `_REFRESH_FORCE_HOURS`, `_POWER_SAVING`, `_REFRESH_FAST`, `_FONT`, `_MENU_IDLE_MINUTES`, `_HEADER_CLOCK`
`sys_text_set(what, text, length)`	one that is text: `SYS_TEXT_TIME_ZONE`, a POSIX TZ string
`sys_time_set(int64_t unix_seconds)`	the clock by hand; fails while it is set from the network
`sys_app_list(ids, capacity)`, `sys_app_title(id, text, capacity)`, `sys_app_remove(id)`	the installed applications by their ids, up to 16
`sys_app_default(void)`, `sys_app_set_default(id)`	what the device shows by itself; 0 is none
`sys_logo(x, y, scale)`, `sys_logo_width(scale)`, `sys_logo_height(scale)`	the word perINKpheraL in its square letters, scale 1 - 5
`sys_do(action)`	`SYS_DO_RESTART`, `_FORGET_NETWORK`, `_FACTORY_RESET`. Ask the user first

23 The SDK library

`mkapp` links the library into every application. Unused functions are removed. The headers are in `sdk/lib`.

`bool` is the standard type from `<stdbool.h>`. Text arguments are null-terminated unless stated otherwise.

app_text.h: short strings without printf

```
typedef struct { char *next; char *last; } app_text_t;      /* a local of the caller's, on the stack */

void    app_text_start(app_text_t *text, char *buffer, int32_t buffer_size);
void    app_text_add(app_text_t *text, const char *more);   /* what does not fit is dropped, between characters */
void    app_text_add_number(app_text_t *text, int32_t value);               /* decimal, "-" when negative */
void    app_text_add_padded(app_text_t *text, int32_t value, int32_t digits);          /* 7, 2 -> "07" */
int32_t app_text_length(const char *text);                  /* of any terminated text: app_string()'s too */
```

The buffer is always null-terminated and stays within its size. The text is complete after each call.

app_values.h: what the host picked

```
bool app_value_text(int32_t index, char *text, int32_t text_size);        /* false: no such value, or no room */
bool app_value_number(int32_t index, int32_t decimals, int32_t *number);  /* "-3.46", 1 -> -35; false: none */
bool app_parse_number(const char *text, int32_t decimals, int32_t *number);    /* the same for a text in hand */
```

These functions work only inside `app_on_values()`. `index` is the position of the path in the `pick` list of the manifest.

app_ui.h: what the buttons do on the kit's screens

```
typedef enum { APP_UI_IGNORED, APP_UI_CHANGED, APP_UI_DONE, APP_UI_CANCELLED } app_ui_result_t;

typedef host_list_t app_list_t;        /* count, selected (-1: facts), top, height, first */
app_ui_result_t app_list_button(app_list_t *list, int32_t event);    /* UP DOWN move; OK done; BACK cancelled */
int32_t         app_list_first(app_list_t *list);                    /* in app_on_draw(), before the rows */

typedef struct { int32_t count; int32_t chosen; bool wraps; } app_picker_t;
app_ui_result_t app_picker_button(app_picker_t *picker, int32_t event);
void            app_picker_draw(const app_picker_t *picker, const char *const *labels);

typedef struct { int32_t year, month, day, hour, minute, has, part, year_first, year_last; } app_date_t;
void            app_date_start(app_date_t *date, int32_t has, int32_t year_first, int32_t year_last);
app_ui_result_t app_date_button(app_date_t *date, int32_t event);    /* OK the next part, done after the last */
void            app_date_draw(const app_date_t *date);
int64_t         app_date_seconds(const app_date_t *date);            /* local: take time_zone_offset() off */
```

BACK on these screens is handled by exporting `app_on_back()` and passing `BTN_BACK_PRESSED` from it to the same button function.

app_alloc.h, json_pick.h, and what is generated

```
void   *app_alloc(int32_t bytes);           /* NULL when so much is not left; not cleared */
int32_t app_alloc_left(void);
int32_t app_alloc_mark(void);               /* and app_alloc_release(mark) gives back all taken since */
void    app_alloc_release(int32_t mark);

void        json_pick_init(json_pick_t *pick, const char *const *paths, int path_count);    /* up to 16 */
bool        json_pick_feed(json_pick_t *pick, const char *bytes, size_t length);      /* a piece as it came */
void        json_pick_finish(json_pick_t *pick);
const char *json_pick_value(const json_pick_t *pick, int index);         /* NULL: the answer had none */

#include "strings.h"      /* made of strings/:  enum { STR_NO_EVENTS, ... };  const char *app_string(int32_t id); */
#include "pictures.h"     /* made of pictures/: static const uint8_t PICTURE_DROP[] = { width, height, bits }; */
```

`strings.h` and `pictures.h` are generated for each application. The build places them first on the include path, so a system header with the same name is not used.

`memcpy`, `memmove`, `memset` and `strlen` are provided because the compiler generates calls to them. They are the whole C library of the platform.

24 Limits

Name	Value	What it bounds
`HOST_API_VERSION`	2	the manifest's `abi`
`MANIFEST_MODULE_MAX_BYTES`	48 KB	`app.wasm`
`MANIFEST_NAME_MAX / _TITLE_MAX`	13 / 23 characters	an application's name and title
`MANIFEST_POLLS_MAX / _HOSTS_MAX / _SETTINGS_MAX`	4 / 4 / 8	what a manifest declares

Name	Value	What it bounds
`MANIFEST_URL_MAX / _SETTING_VALUE_MAX`	511 / 255 bytes	a poll's URL; a setting's value
`HOST_TEXT_MAX`	127 bytes	one text drawn or measured
`HOST_HEADER_TEXT_MAX / HOST_STATUS_TEXT_MAX`	63 / 31 bytes	a header text; a header status
`HOST_KV_KEY_MAX / _VALUE_MAX / _TOTAL_MAX`	15 / 1024 / 4096	a key; a value; an application's whole store
`HOST_FILE_NAME_MAX / HOST_FILES_MAX`	23 / 64 KB	a file's name; all files of an application
`HOST_DATA_CHUNK_MAX`	1024 bytes	a piece of an answer
`HOST_ANSWER_MAX / HOST_ANSWER_PIECES_MAX`	256 KB / 4 MB	an answer picked apart; an answer in pieces
`HOST_VALUE_MAX`	63 bytes	a picked value
`HOST_HTTP_HEADERS_MAX / _BODY_MAX / _PENDING_MAX`	1023 / 512 / 2	a request's headers and body; requests waiting on the device
`HOST_REQUEST_FIRST .. _LAST`	16 .. 255	tags of an application's own requests
`HOST_TIMER_COUNT / HOST_ALARM_COUNT`	4 / 4	per application
`HOST_ALARM_EVERY_MIN .. _MAX`	60 .. 86400 s	a repeating alarm
`HOST_TEXT_ARGUMENTS_MAX`	4	arguments of `text_fill()`
`HOST_QR_TEXT_MAX / HOST_LOG_TEXT_MAX`	200 / 200	a QR code's text; a log line
`HOST_ICON_CELLS / HOST_ICON_CELL_MAX`	11 / 16	an icon's grid; the largest cell in pixels
`strings of one language`	8 KB	the table the host loads

`sdk/host_api.h` is the authoritative contract. If this document and the header disagree, the header is correct.